

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO3: *Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity. (BL2, BL3)*

Activity Tool : Case Study (Medium)
Assessment Tool Weightage: 30%

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Analyze the case: An online shopping platform stores Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price). Identify FDs and normalize to 3NF.	BL2 – Understand	5
2	A hospital maintains Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName). Identify anomalies and normalize to BCNF.	BL3 – Apply	5
3	Case study: A university stores Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade). Analyze FDs and apply 2NF decomposition.	BL3 – Apply	5
4	Given the airline booking case: Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date). Normalize up to BCNF.	BL3 – Apply	5
5	Analyze an HR system schema for a multinational company. Identify all functional and transitive dependencies and normalize to 3NF.	BL2 – Understand	5
6	Case: Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate). Identify MVDs and decompose to 4NF.	BL3 – Apply	5
7	Analyze a telecom billing schema and identify all candidate keys, primary keys, and functional dependencies before normalization.	BL2 – Understand	5
8	A retail inventory system has Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse). Apply complete normalization.	BL3 – Apply	5
9	Study the given poorly normalized schema for a School ERP and propose a fully normalized design up to BCNF with justification.	BL3 – Apply	5
10	Given a movie streaming platform schema, identify join dependencies and decompose to 5NF with lossless-join verification.	BL3 – Apply	5

Code of Conduct:

I Sree B(192524023) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Sree B

1. Analyze the case: An online shopping platform stores Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price). Identify FDs and normalize to 3NF.

To normalize the relation, first identify the functional dependencies (FDs), then decompose the relation until it satisfies **Third Normal Form (3NF)**.

Given Relation

Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price)

Step 1: Identify Functional Dependencies (FDs)

Assume:

- Each order belongs to one customer.
- Each customer has one name.
- Each product has one name and one price.
- An order can contain multiple products.

The functional dependencies are:

1. **OrderID** → **CustID**
2. **CustID** → **CustName**
3. **ProdID** → **ProdName, Price**
4. **(OrderID, ProdID)** → **Qty**

Candidate Key

Since one order can contain many products, the candidate key is:
(OrderID, ProdID)

Step 2: First Normal Form (1NF)

The relation is already in **1NF** because:

- All attributes contain atomic values.
- No repeating groups.

Step 3: Second Normal Form (2NF)

The primary key is **(OrderID, ProdID)**.

Partial dependencies exist:

- **OrderID** → **CustID**
- **ProdID** → **ProdName, Price**

These depend on only part of the composite key, violating 2NF.

Decompose into:

Orders

Orders(OrderID, CustID)

Products

Products(ProdID, ProdName, Price)

OrderDetails

OrderDetails(OrderID, ProdID, Qty)

Step 4: Third Normal Form (3NF)

There is a transitive dependency:

- **OrderID** → **CustID**
- **CustID** → **CustName**

Therefore,

OrderID → **CustName** (transitively)

To remove this dependency:

Customers

Customers(CustID, CustName)

Final 3NF Relations

1. **Customers**
 - **CustID** (PK)

- CustName
- 2. **Orders**
 - OrderID (PK)
 - CustID (FK)
- 3. **Products**
 - ProdID (PK)
 - ProdName
 - Price
- 4. **OrderDetails**
 - OrderID (FK)
 - ProdID (FK)
 - Qty
 - **Primary Key:** (OrderID, ProdID)

Final 3NF Schema

Relation	Attributes	Primary Key
Customers	CustID, CustName	CustID
Orders	OrderID, CustID	OrderID
Products	ProdID, ProdName, Price	ProdID
OrderDetails	OrderID, ProdID, Qty	(OrderID, ProdID)

Benefits of 3NF

- Eliminates data redundancy.
- Removes update, insertion, and deletion anomalies.
- Ensures every non-key attribute depends only on the key, the whole key, and nothing but the key.

2. A hospital maintains Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName). Identify anomalies and normalize to BCNF

Given Relation

Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName)

Assume:

- Each patient has one doctor.
- Each doctor has one name and one specialization.
- Each ward has one ward name.

1. Functional Dependencies

The main functional dependencies are:

1. **PID $\rightarrow$ PName, DocID, WardNo**
2. **DocID $\rightarrow$ DocName, Specialization**
3. **WardNo $\rightarrow$ WardName**

Therefore, the candidate key is:

PID

2. Anomalies

The original table has several problems:

Update Anomaly

If a doctor's specialization changes, it must be updated for **every patient** assigned to that doctor. Missing one row creates inconsistent data.

Insertion Anomaly

A new doctor cannot be stored unless a patient is assigned to that doctor.

Deletion Anomaly

If the only patient assigned to a doctor is deleted, information about that doctor and specialization may also be lost.

Data Redundancy

DocName, Specialization, and WardName are repeatedly stored for multiple patients.

3. Check for BCNF

A relation is in **BCNF** if, for every non-trivial FD:

$X \rightarrow Y$, X must be a superkey.

In the original relation:

- **PID $\rightarrow$...** — PID is a key, so this is valid.
- **DocID $\rightarrow$ DocName, Specialization** — **DocID is not a superkey**, so BCNF is violated.
- **WardNo $\rightarrow$ WardName** — **WardNo is not a superkey**, so BCNF is violated.

Therefore, the original relation is **not in BCNF**.

4. BCNF Decomposition

Patient Table

Patient(PID, PName, DocID, WardNo)

- Primary Key: PID

FD:

PID $\rightarrow$ PName, DocID, WardNo

This relation is in BCNF.

Doctor Table

Doctor(DocID, DocName, Specialization)

- Primary Key: DocID

FD:

DocID $\rightarrow$ DocName, Specialization

DocID is a superkey, so this relation is in BCNF.

Ward Table

Ward(WardNo, WardName)

- Primary Key: WardNo

FD:

WardNo → **WardName**

WardNo is a superkey, so this relation is in BCNF.

Final BCNF Schema

Relation	Attributes	Primary Key
Patient	PID, PName, DocID, WardNo	PID
Doctor	DocID, DocName, Specialization	DocID
Ward	WardNo, WardName	WardNo

Foreign Keys:

- Patient.DocID → Doctor.DocID
- Patient.WardNo → Ward.WardNo

Conclusion

The original **Patient** relation violates BCNF because DocID and WardNo determine other attributes without being superkeys. Decomposing it into **Patient, Doctor, and Ward** removes redundancy and update/insert/delete anomalies, and all three resulting relations satisfy **BCNF**.

3. Case study: A university stores Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade). Analyze FDs and apply 2NF decomposition

Case Study: University Enrollment

Given relation:

Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade)

1. Identify Functional Dependencies

Assume:

- Each student has one name.
- Each course has one course name and is handled by one faculty.
- A student can enroll in many courses.
- A student's grade is specific to a particular course.

The functional dependencies are:

1. **SID → SName**
2. **CourseID → CourseName, Faculty**
3. **(SID, CourseID) → Grade**

2. Candidate Key

A student can take multiple courses, and a course can have multiple students.

Therefore, the candidate key is:

(SID, CourseID)

3. Check for 1NF

The relation is already in **1NF** because all attributes contain atomic values.

However, it is **not in 2NF** because there are **partial dependencies**.

- **SID → SName** — depends only on part of the composite key.
- **CourseID → CourseName, Faculty** — depends only on part of the composite key.

4. 2NF Decomposition

To remove partial dependencies, decompose the relation into three relations.

Student

Student(SID, SName)

- Primary Key: SID
- FD: SID → SName

Course

Course(CourseID, CourseName, Faculty)

- Primary Key: CourseID
- FD: CourseID → CourseName, Faculty

Enrollment

Enrollment(SID, CourseID, Grade)

- Composite Primary Key: (SID, CourseID)
- FD: (SID, CourseID) → Grade

Final 2NF Schema

Relation	Attributes	Primary Key
Student	SID, SName	SID
Course	CourseID, CourseName, Faculty	CourseID
Enrollment	SID, CourseID, Grade	(SID, CourseID)

Foreign Keys:

- Enrollment.SID $\rightarrow$ Student.SID
- Enrollment.CourseID $\rightarrow$ Course.CourseID

Result

The original **Enrollment** table had partial dependencies because SName depended only on SID, while CourseName and Faculty depended only on CourseID. After decomposition, every non-key attribute depends on the **whole key** of its respective relation. Hence, the resulting relations are in **2NF**.

4. Given the airline booking case: Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date). Normalize up to BCNF.

Airline Booking Case

Given relation:

Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date)

Assume:

- Each passenger has one name.
- Each flight has one departure city and arrival city.
- A flight operates on a particular date.
- A passenger gets one seat for a particular flight/date.

1. Functional Dependencies

The important FDs are:

1. **PassID** → **PassName**
2. **(FlightNo, Date)** → **DeptCity, ArrCity**
3. **(FlightNo, Date, PassID)** → **Seat**

Therefore, the candidate key is:

(FlightNo, Date, PassID)

2. First Normal Form (1NF)

The relation is already in **1NF** because all attributes contain atomic values.

3. Second Normal Form (2NF)

The composite key is:

(FlightNo, Date, PassID)

There are partial dependencies:

- **PassID** → **PassName**
- **(FlightNo, Date)** → **DeptCity, ArrCity**

Therefore, the original relation is **not in 2NF**.

Decompose into:

Passenger

Passenger(PassID, PassName)

Primary Key: PassID

Flight

Flight(FlightNo, Date, DeptCity, ArrCity)

Primary Key: (FlightNo, Date)

Booking

Booking(FlightNo, Date, PassID, Seat)

Primary Key: (FlightNo, Date, PassID)

4. Third Normal Form (3NF)

Check the decomposed relations:

Passenger

PassID → **PassName**

No transitive dependency.

Flight

(FlightNo, Date) → **DeptCity, ArrCity**

No transitive dependency.

Booking

(FlightNo, Date, PassID) → **Seat**

No transitive dependency.

Thus, the relations are in **3NF**.

5. BCNF Check

A relation is in **BCNF** if for every non-trivial FD:

$X \rightarrow Y$, X must be a superkey.

Passenger

$\text{PassID} \rightarrow \text{PassName}$

PassID is the primary key $\rightarrow$ **BCNF**.

Flight

$(\text{FlightNo}, \text{Date}) \rightarrow \text{DeptCity}, \text{ArrCity}$

$(\text{FlightNo}, \text{Date})$ is the primary key $\rightarrow$ **BCNF**.

Booking

$(\text{FlightNo}, \text{Date}, \text{PassID}) \rightarrow \text{Seat}$

$(\text{FlightNo}, \text{Date}, \text{PassID})$ is the primary key $\rightarrow$ **BCNF**.

Therefore, all decomposed relations satisfy **BCNF**.

Final BCNF Schema

Relation	Attributes	Primary Key
Passenger	PassID, PassName	PassID
Flight	FlightNo, Date, DeptCity, ArrCity	(FlightNo, Date)
Booking	FlightNo, Date, PassID, Seat	(FlightNo, Date, PassID)

Foreign Keys

- $\text{Booking.PassID} \rightarrow \text{Passenger.PassID}$
- $\text{Booking}(\text{FlightNo}, \text{Date}) \rightarrow \text{Flight}(\text{FlightNo}, \text{Date})$

Conclusion

The original **Booking** relation contains partial dependencies, causing 2NF violations. Decomposing it into **Passenger**, **Flight**, and **Booking** removes the partial dependencies. Since every determinant in the resulting relations is a candidate key, the final schema is in **BCNF**.

5. Analyze an HR system schema for a multinational company. Identify all functional and transitive dependencies and normalize to 3NF.

HR System Schema – 3NF Normalization

Assume the HR system has the following relation:

Employee(EmpID, EmpName, DeptID, DeptName, ManagerID, ManagerName, DesignationID, Designation, Salary)

1. Functional Dependencies

The main functional dependencies are:

1. **EmpID $\rightarrow$ EmpName, DeptID, ManagerID, DesignationID, Salary**
2. **DeptID $\rightarrow$ DeptName, ManagerID**
3. **ManagerID $\rightarrow$ ManagerName**
4. **DesignationID $\rightarrow$ Designation**

Therefore, **EmpID** is the candidate key.

2. Transitive Dependencies

A transitive dependency occurs when a non-key attribute determines another non-key attribute.

Dependency 1

EmpID $\rightarrow$ DeptID

and

DeptID $\rightarrow$ DeptName

Therefore:

EmpID $\rightarrow$ DeptName

So, DeptName is transitively dependent on EmpID.

Dependency 2

EmpID $\rightarrow$ ManagerID

and

ManagerID $\rightarrow$ ManagerName

Therefore:

EmpID $\rightarrow$ ManagerName

So, ManagerName is transitively dependent on EmpID.

Dependency 3

EmpID $\rightarrow$ DesignationID

and

DesignationID $\rightarrow$ Designation

Therefore:

EmpID $\rightarrow$ Designation

So, Designation is transitively dependent on EmpID.

3. Check 1NF

The relation is in **1NF** if all attributes contain atomic values and there are no repeating groups.

Assuming this condition is satisfied, the relation is in **1NF**.

4. Check 2NF

The primary key is **EmpID**, which is a single attribute.

Since there cannot be a partial dependency on a single-attribute key, the relation is automatically in **2NF**.

However, it has **transitive dependencies**, so it is not in 3NF.

5. Decompose into 3NF

Employee Table

Employee(EmpID, EmpName, DeptID, ManagerID, DesignationID, Salary)

- Primary Key: EmpID
- Foreign Keys: DeptID, ManagerID, DesignationID

FD:

EmpID → EmpName, DeptID, ManagerID, DesignationID, Salary

Department Table

Department(DeptID, DeptName, ManagerID)

- Primary Key: DeptID
- Foreign Key: ManagerID

FD:

DeptID → DeptName, ManagerID

Manager Table

Manager(ManagerID, ManagerName)

- Primary Key: ManagerID

FD:

ManagerID → ManagerName

Designation Table

Designation(DesignationID, Designation)

- Primary Key: DesignationID

FD:

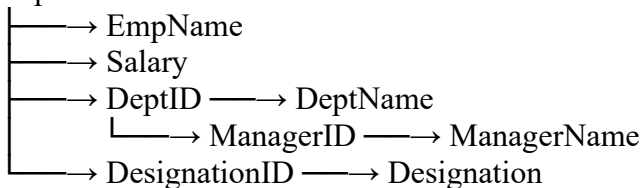
DesignationID → Designation

Final 3NF Schema

Relation	Attributes	Primary Key
Employee	EmpID, EmpName, DeptID, ManagerID, DesignationID, Salary	EmpID
Department	DeptID, DeptName, ManagerID	DeptID
Manager	ManagerID, ManagerName	ManagerID
Designation	DesignationID, Designation	DesignationID

Dependency Structure

EmpID



Why 3NF?

The decomposition removes the transitive dependencies:

- EmpID → DeptName
- EmpID → ManagerName
- EmpID → Designation

Now, in every relation, **non-key attributes depend directly on the primary key and not on another non-key attribute.**

Result: The HR schema is normalized to 3NF, reducing redundancy and preventing insertion, deletion, and update anomalies.

6.Case: Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate). Identify MVDs and decompose to 4NF.

Library Case – 4NF Decomposition

Given relation:

Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate)

Assume:

- Each MemberID identifies one member.
- Each BookID identifies one book.
- A book has one title, author, and genre.
- A member can borrow multiple books.
- The borrowing date is associated with a particular member and book.

1. Functional Dependencies

The main FDs are:

1. **MemberID** → **MemberName**
2. **BookID** → **Title, Author, Genre**
3. **(MemberID, BookID)** → **BorrowDate**

The candidate key is:

(MemberID, BookID)

2. Identify Multivalued Dependencies (MVDs)

A **multivalued dependency (MVD)** exists when one attribute determines multiple independent values of another attribute.

For example:

MemberID →→ **BookID**

This means one member can be associated with multiple books independently.

However, to apply 4NF correctly, we need an independent multivalued relationship. We can represent the case as:

MemberID →→ **BookID**

If the member's borrowing dates are also treated as an independent multivalued fact, then:

MemberID →→ **BorrowDate**

This creates unnecessary combinations of books and dates in the original relation.

3. 4NF Violation

A relation is in 4NF if, for every non-trivial MVD:

X →→ Y, X must be a superkey.

In the original relation:

MemberID →→ **BookID**

But MemberID is **not a superkey** of the complete relation.

Therefore, the relation violates 4NF.

4. Decompose to 4NF

Member Table

Member(MemberID, MemberName)

- Primary Key: MemberID

Book Table

Book(BookID, Title, Author, Genre)

- Primary Key: BookID

Borrowing Table

Borrowing(MemberID, BookID, BorrowDate)

- Primary Key: (MemberID, BookID, BorrowDate)

This separates member information, book information, and borrowing information.

Final 4NF Schema

Relation	Attributes	Primary Key
Member	MemberID, MemberName	MemberID
Book	BookID, Title, Author, Genre	BookID
Borrowing	MemberID, BookID, BorrowDate	(MemberID, BookID, BorrowDate)

Dependency Summary

MemberID $\longrightarrow$ MemberName

BookID $\longrightarrow$ Title

$\longrightarrow$ Author

$\longrightarrow$ Genre

(MemberID, BookID) $\longrightarrow$ BorrowDate

MemberID $\longrightarrow \longrightarrow$ BookID

Conclusion

The original **Library** relation contains a multivalued dependency because a member can be associated with multiple books independently. Since MemberID is not a superkey, this violates 4NF. Decomposing the relation into **Member**, **Book**, and **Borrowing** removes the multivalued dependency and eliminates redundancy.

Final result: The decomposed relations satisfy 4NF.

7. Analyze a telecom billing schema and identify all candidate keys, primary keys, and functional dependencies before normalization

Telecom Billing Schema Analysis

Since no specific schema was provided, consider a typical telecom billing relation:

Billing(BillID, CustomerID, CustomerName, PhoneNo, PlanID, PlanName, PlanRate, CallID, CallDate, CallDuration, CallCharge)

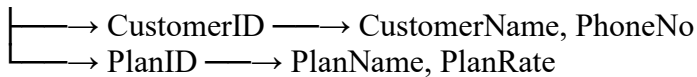
1. Functional Dependencies

The likely functional dependencies are:

1. **BillID** $\rightarrow$ **CustomerID, PlanID**
2. **CustomerID** $\rightarrow$ **CustomerName, PhoneNo**
3. **PlanID** $\rightarrow$ **PlanName, PlanRate**
4. **CallID** $\rightarrow$ **CallDate, CallDuration, CallCharge**
5. **(BillID, CallID)** $\rightarrow$ **all attributes of the billing record**

Thus, the major dependency chain is:

BillID



CallID $\rightarrow$ CallDate, CallDuration, CallCharge

2. Candidate Keys

A **candidate key** is a minimal attribute set that uniquely identifies each tuple.

Under the above assumptions:

Candidate Key = (BillID, CallID)

Why?

- BillID identifies a particular bill.
- A bill can contain multiple call records.
- CallID identifies the individual call.
- Therefore, (BillID, CallID) uniquely identifies a billing record.

Neither BillID nor CallID alone is sufficient.

Candidate Key

{BillID, CallID}

3. Primary Key

The candidate key selected as the primary key is:

Primary Key = (BillID, CallID)

This is a **composite primary key**.

4. Foreign Keys

Likely foreign-key relationships are:

- CustomerID $\rightarrow$ Customer
- PlanID $\rightarrow$ Plan
- BillID $\rightarrow$ Bill
- CallID $\rightarrow$ Call

5. Summary Table

Attribute / Set	Determines	Type
BillID	CustomerID, PlanID	Functional dependency
CustomerID	CustomerName, PhoneNo	Functional dependency
PlanID	PlanName, PlanRate	Functional dependency
CallID	CallDate, CallDuration, CallCharge	Functional dependency
(BillID, CallID)	Complete billing record	Candidate/Primary Key

Final Answer

- **Candidate Key:** (BillID, CallID)
- **Primary Key:** (BillID, CallID)
- **Functional**

BillID	→	CustomerID,	PlanID
CustomerID	→	CustomerName,	PhoneNo
PlanID	→	PlanName,	PlanRate
CallID	→	CallDate,	CallDuration,
(BillID, CallID)	→	CallCharge	

(BillID, CallID) → all attributes

Dependencies:

Note: Candidate keys depend on the exact business rules. If the actual telecom schema has a different definition of BillID or CallID, the keys and FDs should be adjusted accordingly.

8. A retail inventory system has Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse). Apply complete normalization.

Retail Inventory System – Complete Normalization

Given relation:

Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse)

Assume:

- Each PID uniquely identifies a product.
- Each supplier has one supplier name.
- A product belongs to one category.
- A product has one price.
- A product may be stored in one or more warehouses.

1. Functional Dependencies

The main FDs are:

1. **PID $\rightarrow$ PName, SupplierID, Category, Price**
2. **SupplierID $\rightarrow$ SupplierName**
3. **(PID, Warehouse) $\rightarrow$ all attributes of the inventory record**

If a product can exist in multiple warehouses, the candidate key of the original relation is:
(PID, Warehouse)

2. First Normal Form (1NF)

The table is in **1NF** if each attribute contains atomic values.

However, because a product may appear in multiple warehouse records, each product-warehouse combination should be stored as a separate row.

3. Second Normal Form (2NF)

The key is:

(PID, Warehouse)

There are partial dependencies:

- **PID $\rightarrow$ PName, SupplierID, Category, Price**

These attributes depend only on PID, not on the complete composite key (PID, Warehouse).

Therefore, the relation violates **2NF**.

Decompose into:

Product(PID, PName, SupplierID, Category, Price)

Inventory(PID, Warehouse)

4. Third Normal Form (3NF)

In the Product relation:

PID $\rightarrow$ SupplierID

and

SupplierID $\rightarrow$ SupplierName

Therefore:

PID $\rightarrow$ SupplierName

This is a **transitive dependency**.

So, separate supplier information.

Supplier

Supplier(SupplierID, SupplierName)

Product

Product(PID, PName, SupplierID, Category, Price)

Inventory

Inventory(PID, Warehouse)

5. BCNF Check

Supplier

SupplierID $\rightarrow$ SupplierName

SupplierID is the key $\rightarrow$ **BCNF**.

Product

PID $\rightarrow$ PName, SupplierID, Category, Price

PID is the key $\rightarrow$ **BCNF**.

Inventory

(PID, Warehouse) is the key $\rightarrow$ **BCNF**, assuming there are no additional dependencies.

Final Normalized Schema

Relation	Attributes	Primary Key
Supplier	SupplierID, SupplierName	SupplierID
Product	PID, PName, SupplierID, Category, Price	PID
Inventory	PID, Warehouse	(PID, Warehouse)

Foreign Key

- Product.SupplierID $\rightarrow$ Supplier.SupplierID
- Inventory.PID $\rightarrow$ Product.PID

Dependency Diagram

SupplierID $\longrightarrow$ SupplierName

PID $\longrightarrow$ PName
|
├── $\longrightarrow$ SupplierID
├── $\longrightarrow$ Category
└── $\longrightarrow$ Price

(PID, Warehouse) $\longrightarrow$ Inventory record

Normalization Summary

Normal Form	Result	Reason
1NF	✓	Atomic attributes
2NF	✗ initially	Partial dependency on PID
3NF	✗ initially	Transitive dependency through SupplierID
BCNF	✓ after decomposition	Every determinant is a candidate key

Final

Supplier(SupplierID, SupplierName) result:
Product(PID, PName, SupplierID, Category, Price)
Inventory(PID, Warehouse)

This decomposition removes **partial and transitive dependencies**, reduces redundancy, and gives a fully normalized schema up to **BCNF**.

9. Study the given poorly normalized schema for a School ERP and propose a fully normalized design up to BCNF with justification.

Consider a poorly normalized School ERP relation:

SchoolERP(StudentID, StudentName, ClassID, ClassName, TeacherID, TeacherName, SubjectID, SubjectName, Marks)

Step 1: Functional Dependencies

Assume:

- Each student has one name.
- Each class has one class name and one class teacher.
- Each teacher has one name.
- Each subject has one subject name.
- A student's marks are determined by the student and subject.

The main FDs are:

1. **StudentID $\rightarrow$ StudentName, ClassID**
2. **ClassID $\rightarrow$ ClassName, TeacherID**
3. **TeacherID $\rightarrow$ TeacherName**
4. **SubjectID $\rightarrow$ SubjectName**
5. **(StudentID, SubjectID) $\rightarrow$ Marks**

Candidate key:

(StudentID, SubjectID)

Step 2: Problems in the Original Schema

Partial dependencies:

- StudentID $\rightarrow$ StudentName, ClassID
- SubjectID $\rightarrow$ SubjectName

These violate **2NF** because they depend on only part of the composite key.

Transitive dependencies:

- StudentID $\rightarrow$ ClassID $\rightarrow$ ClassName
- StudentID $\rightarrow$ ClassID $\rightarrow$ TeacherID
- TeacherID $\rightarrow$ TeacherName

These violate **3NF**.

Step 3: BCNF Decomposition

Student

Student(StudentID, StudentName, ClassID)

Primary Key: StudentID

Class

Class(ClassID, ClassName, TeacherID)

Primary Key: ClassID

Teacher

Teacher(TeacherID, TeacherName)

Primary Key: TeacherID

Subject

Subject(SubjectID, SubjectName)

Primary Key: SubjectID

StudentSubject

StudentSubject(StudentID, SubjectID, Marks)

Primary Key: (StudentID, SubjectID)

Final BCNF Design

Relation	Primary Key
Student(StudentID, StudentName, ClassID)	StudentID
Class(ClassID, ClassName, TeacherID)	ClassID
Teacher(TeacherID, TeacherName)	TeacherID
Subject(SubjectID, SubjectName)	SubjectID
StudentSubject(StudentID, SubjectID, Marks)	(StudentID, SubjectID)

Each non-trivial determinant is a key in its respective relation, so the decomposition satisfies **BCNF**.

Justification

The decomposition:

- Removes partial dependencies → **2NF**
- Removes transitive dependencies → **3NF**
- Ensures every determinant is a candidate key → **BCNF**
- Reduces data redundancy and update, insertion, and deletion anomalies.

10. Given a movie streaming platform schema, identify join dependencies and decompose to 5NF with lossless-join verification.

Consider a movie streaming platform with:

Streaming(MovieID, PlatformID, ActorID)

Assume a movie can be:

- available on multiple platforms,
- associated with multiple actors,
- and the movie-platform-actor relationships may be represented independently.

1. Identify the Join Dependencies

Suppose the business rules establish three independent relationships:

- **Movie** $\leftrightarrow$ **Platform**
- **Movie** $\leftrightarrow$ **Actor**
- **Platform** $\leftrightarrow$ **Actor**

Then the relation can have the join dependency:

(MovieID, PlatformID), (MovieID, ActorID), (PlatformID, ActorID)

This means the original three-way relationship can be reconstructed from three binary relationships.

The relation may therefore contain unnecessary combinations and redundancy.

2. 5NF Requirement

A relation is in **5NF (PJ/NF)** when every non-trivial join dependency is implied by candidate keys.

If the three-way relation has the above non-trivial join dependency, it violates 5NF.

Decompose into:

MoviePlatform

MoviePlatform(MovieID, PlatformID)

Primary Key:

(MovieID, PlatformID)

MovieActor

MovieActor(MovieID, ActorID)

Primary Key:

(MovieID, ActorID)

PlatformActor

PlatformActor(PlatformID, ActorID)

Primary Key:

(PlatformID, ActorID)

3. Lossless-Join Verification

The original relation can be reconstructed using natural joins:

MoviePlatform $\bowtie$ **MovieActor** $\bowtie$ **PlatformActor**

Therefore:

$R = \pi(\text{MovieID, PlatformID})(R) \bowtie \pi(\text{MovieID, ActorID})(R) \bowtie \pi(\text{PlatformID, ActorID})(R)$

where:

- R = original Streaming relation.

If the decomposition produces exactly the original valid tuples under the stated business rules, the decomposition is **lossless**.

Example

Suppose:

MoviePlatform

MovieID PlatformID

M1 P1

M1 P2

MovieActor

MovieID ActorID

M1 A1

MovieID ActorID

M1 A2

PlatformActor**PlatformID ActorID**

P1 A1

P2 A2

The join reconstructs only the combinations permitted by these relationships.

Final 5NF Schema**Relation****Primary Key**

MoviePlatform(MovieID, PlatformID) (MovieID, PlatformID)

MovieActor(MovieID, ActorID) (MovieID, ActorID)

PlatformActor(PlatformID, ActorID) (PlatformID, ActorID)

Conclusion

The original Streaming(MovieID, PlatformID, ActorID) relation contains a **non-trivial join dependency**. Decomposing it into MoviePlatform, MovieActor, and PlatformActor eliminates redundancy caused by the join dependency.

The decomposition is in 5NF and is lossless, provided the stated business rule truly says that these three pairwise relationships are independent and sufficient to determine valid streaming tuples.